SWISS PYTHON SUMMIT 2026 · 23 OCTOBER · RAPPERSWIL

Before you open that OSS issue: evidence-first triage

How a missing-data problem sent us into the issue tracker of PyTorch, an open-source (OSS) project, and what we built after

Pavel Sulimov

zh aw School of Engineering
InIT Institute of Computer Science

Sergey Eremeykin

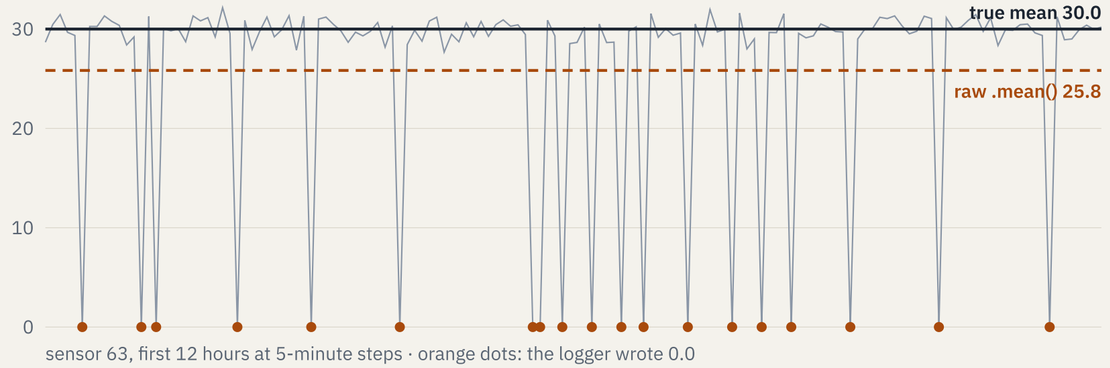
arraxis

```
pip install consta
```

Open source, Apache-2.0
Consta is powered by Arraxis

The task: forecast 64 sensors with gaps in their data

When a sensor goes quiet, the logger writes 0.0. A gap then looks like a real reading, and every statistic and every model trained on it treats it as one.

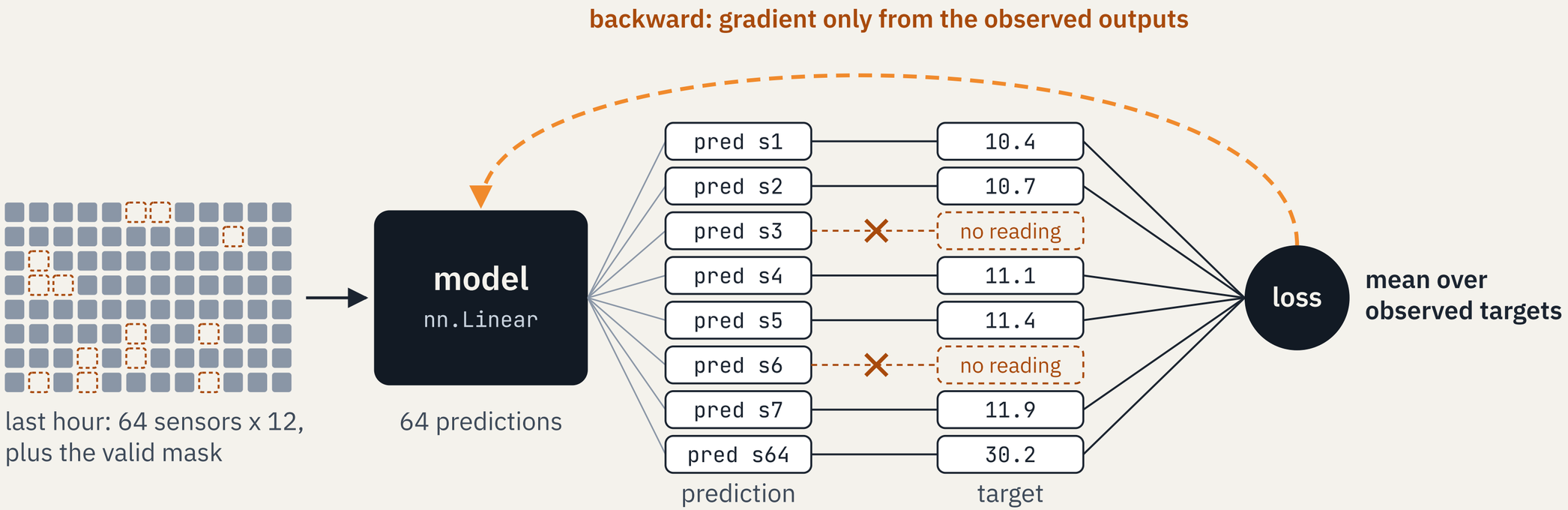


sensor 63, first 12 hours at 5-minute steps · orange dots: the logger wrote 0.0

We need the gaps to count nowhere. Right now even a plain .mean() is off by 3.0 on average, with 288 readings a day and 15% of them missing.

Data: generated, with the shape of an Arraxis sensor project

A model with 64 outputs and missing targets



Train on the logged 0.0 and the model learns that sensors read zero. Average error on unseen data: **-3.01**.

Average the loss over real readings only: average error **-0.24**.

Store the gaps as NaN and hide them with torch.where: the loss looks fine, and every gradient is NaN.

Generated data, one week, 2,004 samples, 14.8% of targets missing

Three ways to write that loss

ZERO-FILLED TARGETS

```
((pred - y)**2).mean()
```

average error -3.01

Runs, converges, and is wrong for every sensor that ever went quiet.

NaN TARGETS + torch.where

```
torch.where(valid,
(pred - y_nan)**2, 0).mean()
```

weight grads: 100% NaN

The forward pass looks fine, but backward multiplies 0 by NaN.

torch.masked

```
y_mt = masked_tensor(y_nan,
valid)
((pred_mt - y_mt)**2).sum()
```

masked: no gradient

The mask travels with the data, through the loss and into the gradient.

The third card is what we wanted: the rule lives in the type. But every masked_tensor() call prints a warning that the API is a prototype.

Could we be the ones to finish a prototype that has waited since 2022?

OUR FIRST QUESTION "Can torch.masked handle our missing sensor readings?"

THE DECISION BEHIND IT "Is reviving torch.masked worth a multi-month upstream contribution, or is a smaller change enough?"

Only the second question tells you what evidence would change your mind.

Like everyone else, we asked a chat first

Q: Should we revive torch.masked upstream?

"A smaller change is enough. MaskedTensor's reduction/broadcasting semantics are genuinely tricky to get right, the API has been effectively unmaintained for years with low adoption…"

Probably right, and without a single link

#89320, open since November 2022: *masked_fill with FloatTensor mask will never mask but fails silently*

Zero commits on torch/masked in the last three months

NestedTensor, the nearest neighbour, is itself *not currently under active development*

Left: claude-sonnet-5, no tools · right: GitHub and docs.pytorch.org

Then we read the tracker ourselves

24

open issues labelled
module: masked operators

Issue	Opened	Title (verbatim)
#89320	2022-11	masked_fill with FloatTensor mask will never mask but fails silently.
#128641	2024-06	Backward is not supported by MaskedTensor's torch.maximum
#150186	2025-03	MaskedTensor 10x slower than Tensor compared with nn.functional.softmax
#169191	2025-11	dim broadcast in masked tensor between mask and data.

Searching for "masked" brings up MPS and TorchScript bugs. The label and the class name find the right ones.

Source: GitHub issues, label "module: masked operators"

The repository is busy, while torch/masked has gone quiet

0
commits, last 3 months

7
commits, last 6 months

22
commits, last 12 months

CODEOWNERS has no entry for torch/masked, so a big pull request here has no assigned reviewer.

Source: git log -- torch/masked and .github/CODEOWNERS in pytorch/pytorch

PyTorch's attention went elsewhere

NestedTensor

Packs sequences of different lengths into one tensor, with offsets instead of padding. No padding means no mask to carry.

"Nested tensors are not currently under active development. Use at your own risk."

`docs.pytorch.org/docs/stable/nested.html`

FlexAttention

You write the mask as a small Python function, mask_mod(b, h, q, kv). torch.compile turns it into one attention kernel that skips the masked blocks.

"Another common attention variant is document masking/jagged sequences."

`pytorch.org/blog/flexattention`

Together they replace the type: ragged batches go to NestedTensor, masked attention to FlexAttention, and the loss, mean and softmax to plain where and masked_fill (next slide).

Sources: docs.pytorch.org and pytorch.org/blog

What we used instead

We needed	Plain PyTorch	MaskedTensor, same task
Masked loss	`y = where(valid, y, 0)` `(((pred - y)**2) * valid).sum() / valid.sum()`	backward fails at the first nn.Linear: "mm is not implemented"
Mean	`where(valid, x, 0).sum(1) / valid.sum(1)`	same result (9.93), 8x slower
Softmax	`x.masked_fill(~valid, -inf).softmax(1)`	same result, 4x slower
Attention	SDPA attn_mask, or FlexAttention mask_mod	not the tool PyTorch maintains for this

Once the targets hold no NaN, the masked loss is two lines. The MaskedTensor error asks you to report it on pytorch/maskedtensor, a repository archived since July 2022.

Data: generated · torch 2.10, CPU, one thread

BEFORE

Revive torch.masked: a multi-month upstream contribution

AFTER

Small fixes a module without an owner can absorb: #89320, #128641

What moved it:

0 commits in 3 months · prototype since 2022, no code owner · masked attention lives in FlexAttention · the loss we needed is two lines of plain PyTorch

2 · BY HAND · 30-60 MINUTES

What we did, as six commands

#	Step	Command
1	What was already said	gh search issues --state closed --match title "MaskedTensor"
2	What shipped	gh search prs --merged "MaskedTensor"
3	Activity on the path	git log --since="6 months ago" -- torch/masked
4	Alternatives, quoted	curl -sL <docs> | rg "not .* active"
5	Process and owners	rg -n "torch/masked" .github/CODEOWNERS
6	Write the brief	claim, quote, link; a teammate opens every link

github.com/sulimovp/consta/blob/main/CHECKLIST.md

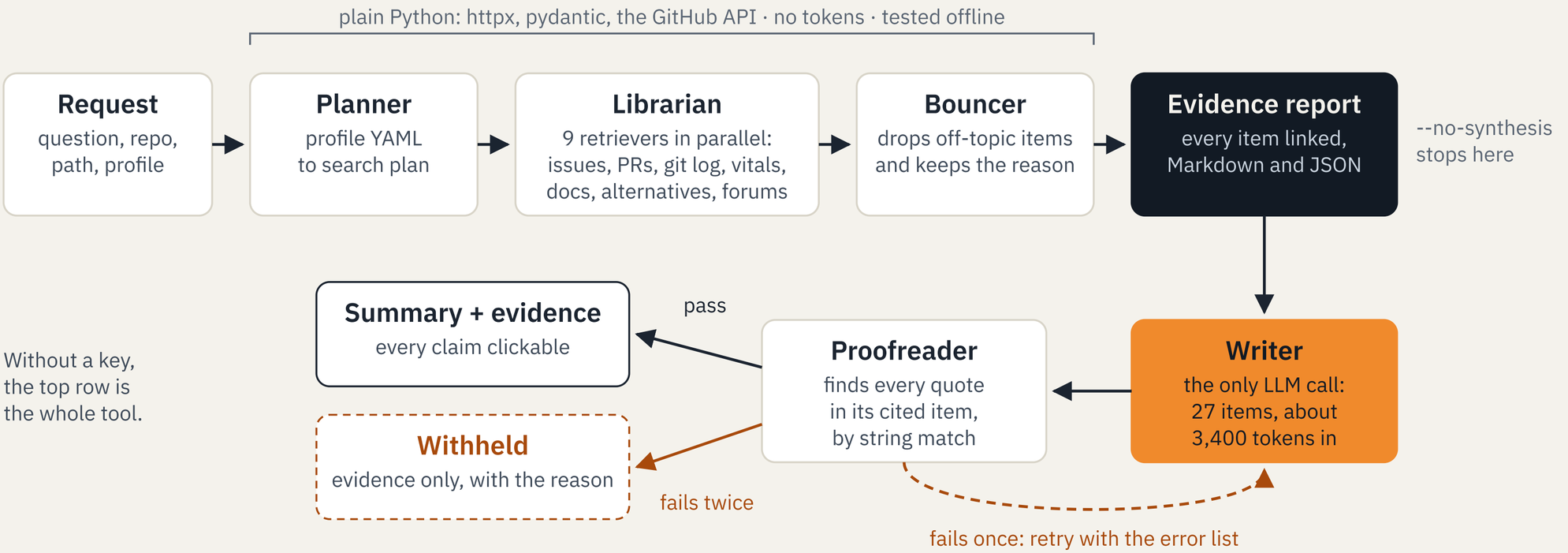
First we looked for someone who had built it

What exists	Answers	Missing for us
CHAOSS, Augur, OpenSSF Scorecard	Is this repo healthy?	One module, not the repo
gha-issue-triage, RepoKeeper	Is this issue a duplicate?	Alternatives outside the repo
oss-scout	Where can I get a PR merged?	Should this feature exist at all?
Elicit, Consensus	What do papers say?	GitHub, docs, forums
A chat or an agent	Anything	Checkable quotes, same answer twice

Each piece exists on its own. We did not find a tool that answers a decision question about one path, with quotes a program checks.

Sources: each project's README and documentation

How Consta works, and where the LLM sits



A YAML profile per ecosystem holds what we learned by hand: label names, synonyms, alternatives with URLs. Write it yourself, or have your own LLM fill in the template and review it.

Every claim carries a quote the code can find

WHAT THE LLM WROTE

The torch/masked module shows declining commit activity with "commits 3/6/12mo = 0/7/23 (falling)" [3]

WHAT ITEM [3] SAYS · vitals-torch-masked

torch/masked: commits 3/6/12mo = 0/7/23 (falling); committers=9; open_issues_total=101

Every paragraph cites, and every [n] points at a real item.

Each quote, at most 15 words, appears word for word in that item.

After one failed retry the summary is withheld, and the evidence stays.

Source: examples/torch-masked.json in the Consta repository

Why not just ask the LLM?

	Plain chat	Agent with web search	Consta
What it reads	its memory	whatever it decides to open	the same searches every run
Time	7 s	61 s, 11 turns	about 30 s, plus one call
Tokens in / out	12,672 / 332	318,238 / 2,692	0 to collect, about 3,400 in to summarise
Who checks the quotes	you	you	a string match, before you see it
Ask twice	a different answer	a different path	the same evidence

The agent does the same searching as our librarian, but pays for it in tokens and decides on its own when to stop.

Take the LLM out and it still works

No model runs at search time, and nothing is trained. It finds things with a profile, fixed searches and plain rules.

1 · PROFILE (YAML)

Written once, by you or drafted by your own LLM: label names, synonyms, alternatives with URLs

2 · FIXED SEARCHES

GitHub issue and label search, git log on the path, CODEOWNERS, the alternatives' pages

3 · RULES

Keep an item only if it names a specific term; labelled issues stay; every drop gets a reason

You keep every item with its link, the commit counts for the path, the alternatives, and what was dropped and why. It needs no API key, costs nothing, and gives the same report for the same input.

You lose the three paragraphs: on torch.masked you read 53 items yourself instead of one page. The decision is yours in both modes.

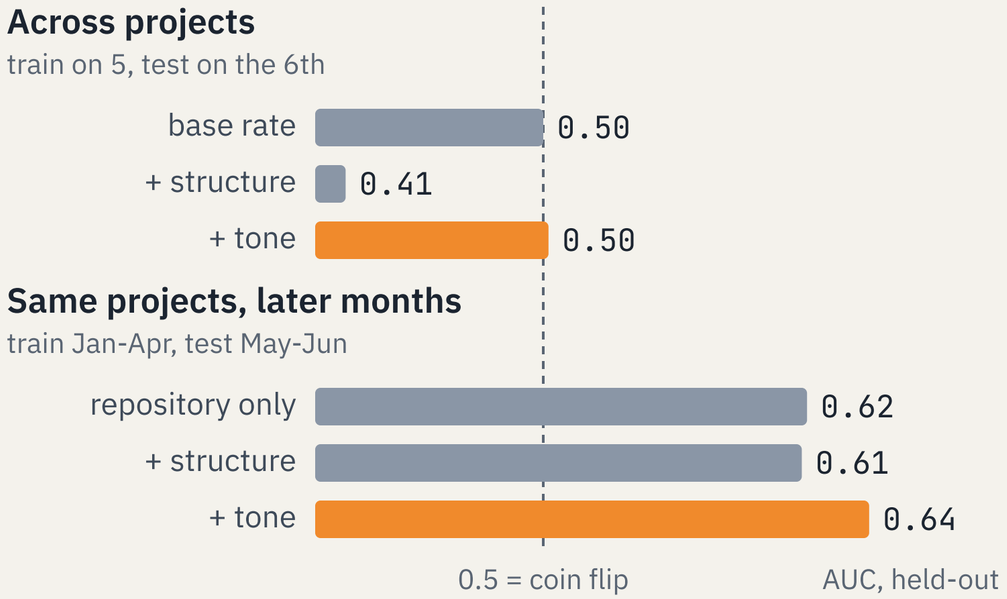
torch.masked run, 17 October 2026: 53 kept, 42 dropped, 30 seconds

Can a model tell which issues will get fixed?

We tried it on 1,663 issues from six projects, opened January to June 2025 and still open after seven days, asking whether each one was fixed within the next 90 days.

In transformers, 54% of those issues were closed by a stale bot, and GitHub records that as "completed". Count it as a fix and the model learns the bot.

Tone helps a little inside a project, but backwards: friendly issues were fixed less often, because feature requests are polite.



A probability this weak does not belong next to a quote, so the report shows counts and dates instead.

Could an LLM read issues for a model?

In August an LLM read 112 issue threads and filled in ten fields each, such as tone, scope and intent. A small model was then meant to predict from those fields whether an issue gets fixed.

Field	What came back	Usable?
affect (tone)	0 on 102 of 105 filled rows	No: a constant
scope	two models barely agree (κ 0.19)	No: noise
maintainer_stance	filled on 7%; the input had no comments	Not measured
intent	two models agree well (κ 0.77)	Yes
severity, specificity	almost always one of two values	Yes, as yes/no

Three fields out of ten were usable, so we dropped the prediction model before we ever trained it.

From PyPI to a report in about thirty seconds

```
$ pip install consta
$ consta assess -q "Is reviving torch.masked worth it?" \
-r pytorch/pytorch -p torch/masked -e pytorch -t 2 \
--no-synthesis -o report.md
```

report.md has the commit counts for the path, quotes from the alternatives' pages, and the 42 items it dropped, each with a reason.

Full output: examples/torch-masked.md in the Consta repository

One environment variable turns the summary on

```
# any of: anthropic, openai, openrouter, huggingface, claude-code
$ export CONSTA_LLM_PROVIDER=huggingface
$ consta assess -q "Is reviving torch.masked worth it?" \
-r pytorch/pytorch -p torch/masked -e pytorch -t 2 -o summary.json
```

You get the same evidence plus three paragraphs on top. Every claim in them has a quote and a number you can click.

Forge one quote, and the summary is thrown out

```
1 · CHECK THE REAL REPORT
$ consta check torch-masked.json
OK: 18 citations, every quote found in its source.
2 · CHANGE ONE QUOTE IN THE SUMMARY, SAVE AS forged.json
commits 3/6/12mo = 0/7/23 (falling) → 9/17/43 (rising)
3 · CHECK THE EDITED COPY
$ consta check forged.json
FAILED: 1 citation error(s). Summary withheld; evidence kept.
- Citation [3] quote 'commits 3/6/12mo = 9/17/43 (rising)'
not found in cited item 'vitals-torch-masked'
```

consta check looks up every quote in the item it cites. One missing quote is enough to drop the summary. The evidence stays.

pandas SparseArray, with no profile

A profile is a YAML file with what we learned by hand about one project: names, labels, alternatives. We wrote them for PyTorch, NumPy, scikit-learn and Apertus. Nobody wrote one for pandas.

TOP OF THE REPORT

[1] PERF: pandas takes 8-9 seconds to import in a new environment
[9] BUG: Adding label argument to plot() removes error bars
[10] BUG: decimal.Decimal treated as string in DataFrame.to_excel()

`dropped as off-topic: 0 · alternatives: none · open questions: none`

Without a profile the off-topic filter does not run, so whatever GitHub search returns for the question goes straight into the report.

The fix: write a pandas profile, or ask your own LLM to fill in the template. Next for us: say "I do not know this module" instead of filling the page.

Apertus: different venue, thinner evidence

Does swiss-ai/apertus-format still earn its place now that Apertus 1.5 ships native tool use and a thinking mode?

Where people talk

Hugging Face Hub discussions: 7 on 1.5, 33 still on the predecessor model

What decides it

One draft PR, "Changes For Upstreaming to transformers", and a docs page that says "Coming soon"

The answer

Not redundant yet. It changes the day that draft pull request is merged.

The method works outside GitHub: supporting the Hugging Face Hub took one profile and one retriever. The evidence there is thinner, and a good report says so.

A real quote can carry a wrong sentence

ANOTHER MODEL WROTE · PASSED THE CHECK

"...with only **0 commits in the last 6 months** and 7 in the last 12 months..." "commits 3/6/12mo = 0/7/23 (falling)" [3]

The quote is genuine, but the sentence misreads it: 6 months is 7, not 0.

Another model wrote "Documentation is still absent" and quoted #89734. The title is real, but the issue was closed in 2022.

The check proves the quote exists. Whether the sentence follows from it is still for you to read, so open the links.

Before you open an issue or start a month-long pull request

1

Write the question as a decision, with its cost

2

Measure the directory you will touch, not the repository

3

Accept a claim only with a link and a sentence you can find there

`github.com/sulimovp/consta/blob/main/CHECKLIST.md` `pip install consta`

QUESTIONS





Thank you

`github.com/sulimovp/consta/blob/main/CHECKLIST.md`

```
pip install consta
```

Consta is powered by Arraxis.



Pavel Sulimov

`sulimov@arraxis.com`



Sergey Eremeykin

`eremeykin@arraxis.com`